



Εθνικό Μετσόβιο Πολυτεχνείο

**Σχολή Ηλεκτρολόγων Μηχανικών & Μηχανικών
Υπολογιστών**

Εξάμηνο 6^ο: Βάσεις Δεδομένων

Περίληψη

Η παρούσα εργαστηριακή αναφορά αφορά την εξαμηνιαία εργασία στα πλαίσια του μαθήματος Βάσεις Δεδομένων. Στην άσκηση αυτή σχεδιάσαμε και υλοποιήσαμε ένα σύστημα αποθήκευσης και διαχείρισης πληροφοριών που απαιτούνται για την εύρυθμη λειτουργία ενός σύγχρονου Νοσοκομείου.

Παναγιώτης Παπαγγελής (03123444)

Βασίλης Τσιώπος (03123167)

Μάρκος-Μηνάς Μαργαρίτης (03123906)

Πίνακας Περιεχομένων

Πίνακας Περιεχομένων.....	2
Σχεδιασμός και υλοποίηση βάσης δεδομένων.....	3
Σχεσιακό διάγραμμα (Relational Model).....	5
Περιορισμοί (Constraints).....	6
Πίνακας ΠΡΟΣΩΠΙΚΟ.....	6
Πίνακας ΙΑΤΡΟΙ.....	6
Πίνακας ΝΟΣΗΛΕΥΤΕΣ.....	6
Πίνακας ΤΜΗΜΑ & ΚΛΙΝΕΣ.....	6
Πίνακας ΑΣΘΕΝΕΙΣ.....	6
Πίνακας ΤΜΗΜΑ_ΕΠΕΙΓΟΝΤΩΝ_ΠΕΡΙΣΤΑΤΙΚΩΝ (ΤΕΠ).....	6
Πίνακας ΝΟΣΗΛΕΙΑ & ΔΙΑΓΝΩΣΕΙΣ.....	7
Πίνακες ΙΑΤΡΙΚΩΝ ΠΡΑΞΕΩΝ & ΕΞΕΤΑΣΕΩΝ.....	7
Πίνακας ΣΥΝΤΑΓΟΓΡΑΦΗΣΗ.....	7
Πίνακας ΒΑΡΔΙΕΣ.....	7
Πίνακες ΑΞΙΟΛΟΓΗΣΕΩΝ & ΚΟΣΤΟΛΟΓΗΣΗΣ (ΚΕΝ).....	7
Triggers:.....	7
1. Περιορισμοί Εποπτείας Ιατρών.....	8
2. Διαχείριση Βαρδιών & Ωραρίων.....	8
3. Διαχείριση Ιατρικών Πράξεων & Χώρων.....	8
4. Αξιολογήσεις Υπηρεσιών Υγείας.....	8
5. Ασφάλεια Συνταγογράφησης & Φαρμάκων.....	9
Ευρετήρια (Indexes).....	9
Fake Data.....	13
Κοστολόγηση και Φάρμακα.....	15
Ανεξαρτησία του πίνακα ΦΩΤΟΓΡΑΦΙΕΣ.....	15
(α) Αποτέλεσμα EXPLAIN.....	15
(β) Εναλλακτική έκδοση με FORCE INDEX.....	16
(γ) Σύγκριση εκτιμώμενου κόστους και πραγματικού χρόνου εκτέλεσης.....	17
(δ) Ανάλυση Συμπερασμάτων.....	17
Q06 — Ανάλυση EXPLAIN και FORCE INDEX.....	17
(α) Αποτέλεσμα EXPLAIN.....	17
(β) Εναλλακτική έκδοση με FORCE INDEX.....	18
(γ) Σύγκριση εκτιμώμενου κόστους και πραγματικού χρόνου εκτέλεσης.....	19
(δ) Ανάλυση Συμπερασμάτων.....	19
Διαχείριση Μεγάλου Όγκου Δεδομένων & Βελτιστοποίηση Εισαγωγής.....	20
AI/LLMS.....	21

Ασθενείς - Προσωπικό: Κάθε ασθενής μπορεί να νοσηλευτεί πολλές φορές. Η σχέση αυτή λοιπόν είναι 1:N. Ένας ασθενής μπορεί, στο πέρασμα του χρόνου, να εισαχθεί στο νοσοκομείο και να πραγματοποιήσει πολλές νοσηλείες. Αντίστροφα, μία συγκεκριμένη καταγραφή νοσηλείας (με τον μοναδικό της κωδικό) αφορά αποκλειστικά έναν και μόνο ασθενή.

Ασθενείς – Στοιχεία_Οικείων_Ατόμων : Κάθε ασθενής μπορεί να έχει καταχωρημένα πολλά οικεία πρόσωπα (στοιχεία οικείων ατόμων), και κάθε οικείο πρόσωπο καταχωρείται για ακριβώς έναν ασθενή. Συνεπώς, η σχέση αυτή είναι 1:N από ασθενείς προς στοιχεία οικείων ατόμων και παρουσιάζει ολική συμμετοχή (total participation) από την πλευρά των οικείων ατόμων, καθώς κανένα οικείο πρόσωπο δεν καταχωρείται στο σύστημα αν δεν σχετίζεται με καθορισμένο ασθενή.

Ιατροί – Τμήμα : Κάθε ιατρός μπορεί να προσφέρει τις υπηρεσίες του σε πολλά τμήματα (τμήμα), και κάθε τμήμα στελεχώνεται από πολλούς ιατρούς. Συνεπώς, η σχέση αυτή είναι M:N (Πολλά-προς-Πολλά) μεταξύ ιατρών και τμημάτων και παρουσιάζει ολική συμμετοχή (total participation) από την πλευρά των ιατρών, καθώς κάθε ιατρός πρέπει να ανήκει σε τουλάχιστον ένα καθορισμένο τμήμα για να εργάζεται στο νοσοκομείο.

• **Τμήμα – Κλίνες :** Κάθε τμήμα περιλαμβάνει πολλές κλίνες , και κάθε κλίνη τοποθετείται σε ακριβώς ένα τμήμα. Συνεπώς, η σχέση αυτή είναι 1:N από τμήμα προς κλίνες και παρουσιάζει ολική συμμετοχή (total participation) από την πλευρά των κλινών, καθώς κάθε κλίνη πρέπει να ανήκει σε καθορισμένο τμήμα του νοσοκομείου.

• **Τμήμα – Νοσηλευτές :** Κάθε τμήμα χρειάζεται νοσηλευτικό προσωπικό, δηλαδή απασχολεί πολλούς νοσηλευτές (νοσηλευτές), και κάθε νοσηλεύτης εργάζεται σε ακριβώς ένα τμήμα. Συνεπώς, η σχέση αυτή είναι 1:N από τμήμα προς νοσηλευτές και παρουσιάζει ολική συμμετοχή (total participation) από την πλευρά των νοσηλευτών, καθώς κάθε νοσηλεύτης πρέπει να εργάζεται σε καθορισμένο τμήμα.

• **Κλίνες – Νοσηλεία :** Κάθε κλίνη μπορεί να φιλοξενήσει πολλές νοσηλείες (νοσηλεία) σε διαφορετικές χρονικές στιγμές, και κάθε νοσηλεία αφορά (δεσμεύει) ακριβώς μία κλίνη. Συνεπώς, η σχέση αυτή είναι 1:N από κλίνες προς νοσηλεία και παρουσιάζει ολική συμμετοχή (total participation) από την πλευρά της νοσηλείας, καθώς κάθε εγγραφή νοσηλείας πρέπει να δεσμεύει υποχρεωτικά μια καθορισμένη κλίνη του νοσοκομείου.

• **Ασθενείς – Τμήμα_Επειγόντων_Περιστατικών (ΤΕΠ) :** Κάθε ασθενής μπορεί να έχει πολλαπλές καταγεγραμμένες επισκέψεις στο τμήμα επειγόντων περιστατικών (ΤΕΠ), και κάθε περιστατικό στο ΤΕΠ αφορά ακριβώς έναν ασθενή. Συνεπώς, η σχέση αυτή είναι 1:N από ασθενείς προς ΤΕΠ και παρουσιάζει ολική συμμετοχή (total participation) από την πλευρά του ΤΕΠ, καθώς κάθε καταγραφή επείγοντος περιστατικού πρέπει να συνδέεται με έναν καθορισμένο ασθενή.

• **Ασθενείς – Τμήμα_Επειγόντων_Περιστατικών (ΤΕΠ) :** Κάθε ασθενής μπορεί να έχει πολλαπλές καταγεγραμμένες επισκέψεις στο τμήμα επειγόντων περιστατικών (ΤΕΠ), και κάθε περιστατικό στο ΤΕΠ αφορά ακριβώς έναν ασθενή. Συνεπώς, η σχέση αυτή είναι 1:N από ασθενείς προς ΤΕΠ και παρουσιάζει ολική συμμετοχή (total participation) από την πλευρά του ΤΕΠ, καθώς κάθε καταγραφή επείγοντος περιστατικού πρέπει να συνδέεται με έναν καθορισμένο ασθενή.

• **Προσωπικό – Βάρδιες :** Κάθε μέλος του προσωπικού μπορεί να συμμετέχει σε πολλές βάρδιες (βάρδιες), και κάθε βάρδια στελεχώνεται από πολλά μέλη του προσωπικού. Συνεπώς, η σχέση αυτή είναι M:N (Πολλά-προς-Πολλά) και παρουσιάζει ολική συμμετοχή

Σχηματικό διάγραμμα (Relational Model)

[illegible]

Περιορισμοί (Constraints)

Αρχικά, η θεμελιώδης δομή της βάσης, που περιλαμβάνει τους περιορισμούς για τα Primary/Foreign keys, καθορίζεται στο προηγούμενο Σχεσιακό Μοντέλο. Στη συνέχεια, προκειμένου να προστατευτεί η βάση δεδομένων από λανθασμένες καταχωρήσεις και να διατηρηθεί η ακεραιότητα πεδίου τιμών, προστέθηκαν οι εξής περιορισμοί ελέγχου (CHECK constraints):

Πίνακας ΠΡΟΣΩΠΙΚΟ

- CHECK (ΗΛΙΚΙΑ $\geq$ 18): Το προσωπικό του νοσοκομείου πρέπει να είναι ενήλικο (τουλάχιστον 18 ετών).
- CHECK (ΤΥΠΟΣ_ΠΡΟΣΩΠΙΚΟΥ IN ('Ιατρός', 'Νοσηλεύτης', 'Διοικητικός')): Επιτρεπόμενοι ρόλοι-τύποι για το προσωπικό του νοσοκομείου.

Πίνακας ΙΑΤΡΟΙ

- CHECK (ΒΑΘΜΙΔΑ IN ('Ειδικευόμενος', 'Επιμελητής Β', 'Επιμελητής Α', 'Διευθυντής')): Επιτρεπόμενες ιεραρχικές βαθμίδες για τους ιατρούς.

Πίνακας ΝΟΣΗΛΕΥΤΕΣ

- CHECK (ΒΑΘΜΙΔΑ IN ('Βοηθός Νοσηλεύτη', 'Νοσηλεύτης', 'Προϊστάμενος')): Επιτρεπόμενες ιεραρχικές βαθμίδες για το νοσηλευτικό προσωπικό.

Πίνακας ΤΜΗΜΑ & ΚΛΙΝΕΣ

- CHECK (ΑΡΙΘΜΟΣ_ΚΛΙΝΩΝ $>$ 0): Ένα τμήμα πρέπει να έχει τουλάχιστον μία διαθέσιμη κλίνη.
- CHECK (ΤΥΠΟΣ IN ('ΜΕΘ', 'Μονόκλινο', 'Πολύκλινο', 'Δίκλινο')): Επιτρεπόμενοι τύποι δωματίων/κλινών.
- CHECK (ΚΑΤΑΣΤΑΣΗ IN ('Διαθέσιμη', 'Κατειλημμένη', 'Υπό Συντήρηση')): Επιτρεπόμενες καταστάσεις λειτουργίας μιας κλίνης.

Πίνακας ΑΣΘΕΝΕΙΣ

- CHECK (ΗΛΙΚΙΑ $\geq$ 0): Η ηλικία του ασθενή δεν μπορεί να είναι αρνητική (επιτρέπεται 0 για βρέφη).
- CHECK (ΦΥΛΟ IN ('Ανδρας', 'Γυναίκα')): Επιτρεπόμενες τιμές για το φύλο του ασθενή.
- CHECK (ΒΑΡΟΣ $>$ 0): Το βάρος του ασθενή πρέπει να είναι αυστηρά θετικός αριθμός.
- CHECK (ΥΨΟΣ $>$ 0): Το ύψος του ασθενή πρέπει να είναι αυστηρά θετικός αριθμός.

Πίνακας ΤΜΗΜΑ_ΕΠΕΙΓΟΝΤΩΝ_ΠΕΡΙΣΤΑΤΙΚΩΝ (ΤΕΠ)

- CHECK (ΕΠΙΠΕΔΟ_ΕΠΕΙΓΟΝΤΟΣ BETWEEN 1 AND 5): Η διαλογή του περιστατικού (triage) γίνεται σε κλίμακα από 1 (λιγότερο επείγον) έως 5 (άμεσα απειλητικό για τη ζωή).

- CHECK (ΠΟΡΙΣΜΑ IN (0,1)): Το πόρισμα καταγράφεται ως λογική τιμή, όπου 1 υποδηλώνει εισαγωγή στο νοσοκομείο και 0 εξιτήριο.

Πίνακας ΝΟΣΗΛΕΙΑ & ΔΙΑΓΝΩΣΕΙΣ

- CHECK (ΚΟΣΤΟΣ_ΝΟΣΗΛΕΙΑΣ >= 0): Το συνολικό κόστος της νοσηλείας δεν μπορεί να είναι αρνητικό.
- CHECK (ΗΜΕΡΟΜΗΝΙΑ_ΕΞΟΔΟΥ IS NULL OR ΗΜΕΡΟΜΗΝΙΑ_ΕΞΟΔΟΥ >= ΗΜΕΡΟΜΗΝΙΑ_ΕΙΣΑΓΩΓΗΣ): Αν ο ασθενής έχει πάρει εξιτήριο, η ημερομηνία εξόδου πρέπει να είναι ίση ή μεταγενέστερη της ημερομηνίας εισαγωγής.
- CHECK (ΤΥΠΟΣ IN ('Εισόδου','Εξόδου')): Μια διάγνωση (ICD10) για μια νοσηλεία μπορεί να είναι μόνο είτε εισόδου (αρχική) είτε εξόδου (τελική).

Πίνακες ΙΑΤΡΙΚΩΝ ΠΡΑΞΕΩΝ & ΕΞΕΤΑΣΕΩΝ

- CHECK (ΚΑΤΗΓΟΡΙΑ IN ('ΠΡΑΞΕΙΣ ΑΝΑΙΣΘΗΣΙΑΣ', ... , 'ΠΡΑΞΕΙΣ ΒΙΟΠΑΘΟΛΟΓΙΑΣ', κ.λπ.)): Επιτρεπόμενες ιατρικές κατηγορίες στις οποίες μπορεί να ανήκει μια πράξη.
- CHECK (ΔΙΑΡΚΕΙΑ > 0): Η διάρκεια μιας ιατρικής πράξης (σε λεπτά) πρέπει να είναι αυστηρά θετικός αριθμός.
- CHECK (ΚΟΣΤΟΣ >= 0): Το κόστος μιας εξέτασης ή ιατρικής πράξης δεν μπορεί να είναι αρνητικό.

Πίνακας ΣΥΝΤΑΓΟΓΡΑΦΗΣΗ

- CHECK (ΗΜΕΡΟΜΗΝΙΑ_ΛΗΞΗΣ IS NULL OR ΗΜΕΡΟΜΗΝΙΑ_ΛΗΞΗΣ >= ΗΜΕΡΟΜΗΝΙΑ_ΕΝΑΡΞΗΣ): Αν υπάρχει καθορισμένη λήξη για την αγωγή, αυτή πρέπει να είναι ίδια ή μεταγενέστερη της ημερομηνίας έναρξης.

Πίνακας ΒΑΡΔΙΕΣ

- CHECK (ΤΥΠΟΣ IN ('Πρωινή', 'Απογευματινή', 'Νυχτερινή')): Επιτρεπόμενοι τύποι βαρδιών για το προσωπικό.

Πίνακες ΑΞΙΟΛΟΓΗΣΕΩΝ & ΚΟΣΤΟΛΟΓΗΣΗΣ (ΚΕΝ)

- CHECK (ΠΟΙΟΤΗΤΑ_... BETWEEN 1 AND 5): (Εφαρμόζεται στα πεδία: Ποιότητα Νοσηλευτικής Φροντίδας, Καθαριότητα, Φαγητό, Συνολική Εμπειρία, Ποιότητα Ιατρικής Φροντίδας). Οι βαθμολογίες αξιολόγησης πρέπει να κυμαίνονται από 1 (Κάκιση) έως 5 (Άριστη).
- CHECK (ΚΟΣΤΟΣ_ΑΝΑ_ΚΕΝ > 0) & CHECK (ΜΔΝ > 0): Το κόστος ανά Κλειστό Ενοποιημένο Νοσήλιο (ΚΕΝ) και η Μέση Διάρκεια Νοσηλείας (ΜΔΝ) πρέπει να είναι θετικοί αριθμοί.

Triggers:

Ένα Trigger είναι ένα ειδικό τμήμα κώδικα SQL που εκτελείται αυτόματα από το σύστημα της βάσης δεδομένων όταν συμβαίνει ένα συγκεκριμένο γεγονός (π.χ., INSERT, UPDATE, ή

DELETE) σε έναν πίνακα. Χρησιμοποιείται για να επιβάλει περίπλοκους κανόνες και περιορισμούς που δεν μπορούν να ελεγχθούν με τα απλά CHECK constraints (π.χ. όταν χρειάζεται έλεγχος δεδομένων από διαφορετικούς πίνακες).

1. Περιορισμοί Εποπτείας Ιατρών

- **Έλεγχος Ύπαρξης Επόπτη (trg_iatros_epopteia_insert/update):** Διασφαλίζει ότι κάθε "Ειδικευόμενος" ιατρός έχει υποχρεωτικά ανατεθειμένο επόπτη κατά την εισαγωγή ή ενημέρωση των στοιχείων του, ενώ απαγορεύει αυστηρά την ανάθεση επόπτη σε ιατρούς με βαθμίδα "Διευθυντή".
- **Αποτροπή Κυκλικής Εποπτείας (trg_iatros_no_cycle_insert/update):** Αναλύει αναδρομικά (έως 200 επίπεδα) την αλυσίδα εποπτείας για να αποτρέψει την κυκλική αλυσίδα εποπτείας (πχ Α επόπτης του Β, Β επόπτης του Α).

2. Διαχείριση Βαρδιών & Ωραρίων

- **Μηνιαίο Όριο Βαρδιών (trg_omada_monthly_limit):** Ελέγχει το πλήθος των βαρδιών που έχει αναλάβει ένα μέλος του προσωπικού μέσα στον τρέχοντα μήνα και μπλοκάρει την ανάθεση αν ξεπεραστεί το ανώτατο όριο (15 για Ιατρούς, 20 για Νοσηλευτές, 25 για Διοικητικούς).
- **Ελάχιστος Χρόνος Ανάπαυσης (trg_omada_8hour_rest):** Υπολογίζει την ώρα λήξης της προηγούμενης βάρδιας ενός υπαλλήλου και αποτρέπει την ανάθεση νέας βάρδιας αν δεν μεσολαβεί το νόμιμο διάστημα ανάπαυσης των 8 ωρών.
- **Όριο Συνεχόμενων Νυχτερινών (trg_omada_consecutive_nights):** Απαγορεύει την ανάθεση νυχτερινής βάρδιας σε υπάλληλο εάν αυτός έχει ήδη εργαστεί σε νυχτερινή βάρδια τις 3 αμέσως προηγούμενες συνεχόμενες ημέρες.
- **Υποχρεωτική Επίβλεψη Ειδικευόμενου (trg_omada_intern_supervision):** Κατά την τοποθέτηση ενός Ειδικευόμενου σε μια βάρδια, ο trigger διασφαλίζει ότι στην ίδια ομάδα εφημερίας έχει ήδη ανατεθεί τουλάχιστον ένας ιατρός ανώτερης βαθμίδας (Επιμελητής Α΄ ή Διευθυντής).

3. Διαχείριση Ιατρικών Πράξεων & Χώρων

- **Αποφυγή Ταυτόχρονων Επεμβάσεων (trg_no_concurrent_operation):** Υπολογίζει τον χρόνο λήξης μιας ιατρικής πράξης (βάσει της προβλεπόμενης διάρκειάς της) και αποτρέπει την καταχώρηση νέας επέμβασης εάν υπάρχει χρονική αλληλεπικάλυψη είτε για την ίδια αίθουσα (χώρος κατειλημμένος) είτε για τον ίδιο ιατρό (αδυναμία ταυτόχρονης παρουσίας).

4. Αξιολογήσεις Υπηρεσιών Υγείας

- **Αξιολόγηση Μόνο Μετά το Εξιτήριο (trg_eval_admission_complete):** Επιτρέπει την αξιολόγηση μιας νοσηλείας από τον ασθενή μόνο εφόσον έχει καταγραφεί "Ημερομηνία Εξόδου" (δηλαδή η νοσηλεία έχει ολοκληρωθεί).
- **Έγκυρη Αξιολόγηση Ιατρού (trg_eval_doctor_prescribed):** Προϋποθέτει ολοκληρωμένη νοσηλεία και ταυτόχρονα ελέγχει τον πίνακα Συνταγογράφησης, επιτρέποντας την αξιολόγηση ενός ιατρού από τον ασθενή μόνο εάν ο συγκεκριμένος

ιατρός είχε ενεργή συμμετοχή (έγραψε φάρμακα) κατά τη διάρκεια της εν λόγω νοσηλείας.

5. Ασφάλεια Συνταγογράφησης & Φαρμάκων

- **Αποτροπή Αλλεργίας (trg_no_allergy_prescription):** Πριν από κάθε νέα καταχώρηση συνταγής, ο trigger ελέγχει αν το συνταγογραφούμενο φάρμακο περιέχει δραστικές ουσίες (μέσω του πίνακα ΦΑΡΜΑΚΑ_ΑΡΘΡΟ_57) στις οποίες ο συγκεκριμένος ασθενής έχει δηλωμένη αλλεργία, μπλοκάροντας άμεσα τη διαδικασία εάν βρεθεί αντιστοιχία.

Ευρετήρια (Indexes)

Τα Ευρετήρια (Indexes) είναι δομές δεδομένων που δημιουργούνται σε μία ή περισσότερες στήλες ενός πίνακα. Λειτουργούν ακριβώς όπως το ευρετήριο στο τέλος ενός βιβλίου: επιτρέπουν στο Σύστημα Διαχείρισης Βάσεων Δεδομένων (DBMS) να εντοπίζει αστραπιαία τις ζητούμενες εγγραφές χωρίς να χρειάζεται να διαβάσει σειριακά ολόκληρο τον πίνακα. Η δημιουργία τους είναι κρίσιμη για την επιτάχυνση των ερωτημάτων ανάγνωσης (SELECT), ειδικά σε πίνακες με τεράστιο όγκο δεδομένων. Ωστόσο, χρησιμοποιούνται στοχευμένα, καθώς καταλαμβάνουν επιπλέον αποθηκευτικό χώρο και προσθέτουν ένα μικρό υπολογιστικό κόστος κατά τις λειτουργίες εγγραφής (INSERT, UPDATE, DELETE), αφού το ευρετήριο πρέπει να ενημερώνεται κάθε φορά που αλλάζουν τα δεδομένα.

Να σημειωθεί, ότι η MySQL η οποία χρησιμοποιήσαμε, έχει ως default engine InnoDB και φτιάχνει αυτόματα index σε όλα τα PRIMARY KEYS και σε όλα τα FOREIGN KEYS. Οπότε δεν χρειάζεται να φτιάξουμε εμείς indexes για όλα αυτά τα κλειδιά. Τα ευρετήρια που δημιουργήσαμε για τη βάση μας, εξηγούνται παρακάτω:

1. CREATE INDEX idx_iatros_eidikotita ON IATPOI (ΕΙΔΙΚΟΤΗΤΑ); Στα ερωτήματα (όπως τα Q02 και Q05) όπου μας ζητείται να βρούμε, να ομαδοποιήσουμε ή να καταμετρήσουμε ιατρούς βάσει της ειδικότητάς τους, αυτό το ευρετήριο μας είναι πολύ χρήσιμο. Επιταχύνει τις πράξεις αναζήτησης και το φιλτράρισμα, καθώς εντοπίζει γρήγορα μόνο τους ιατρούς που ανήκουν στη ζητούμενη ειδικότητα (π.χ. WHERE ΕΙΔΙΚΟΤΗΤΑ = 'Καρδιολόγος'), χωρίς να χρειάζεται να σαρώνει ολόκληρο τον πίνακα IATPOI.

2. CREATE INDEX idx_prosop_ilikia ON ΠΡΟΣΩΠΙΚΟ (ΗΛΙΚΙΑ); Αυτό το ευρετήριο το δημιουργήσαμε συγκεκριμένα για ερωτήματα (όπως το Q05) όπου πρέπει να βρούμε νέους ιατρούς ή μέλη του προσωπικού υπό τη συνθήκη ότι έχουν ηλικία κάτω από ένα συγκεκριμένο όριο (π.χ. κάτω των 35 ετών). Το index κάνει πιο αποδοτική την αναζήτηση της ηλικίας, επιταχύνοντας δραματικά τα φίλτρα σύγκρισης (WHERE ΗΛΙΚΙΑ < 35).

3. CREATE INDEX idx_praxis_date ON ΙΑΤΡΙΚΕΣ_ΠΡΑΞΕΙΣ_ΑΣΘΕΝΗ (ΗΜΕΡΟΜΗΝΙΑ_ΩΡΑ); Το συγκεκριμένο ευρετήριο είναι πολύ σημαντικό καθώς σε ερωτήματα (όπως το Q11) μας ζητείται να αναζητήσουμε ιατρικές πράξεις και επεμβάσεις που πραγματοποιήθηκαν μέσα στο τρέχον έτος. Με το να έχουμε ένα index στην ημερομηνία της πράξης, επιταχύνουμε τον έλεγχο (WHERE YEAR(ΗΜΕΡΟΜΗΝΙΑ_ΩΡΑ) = 2024) και

ανακτούμε ταχύτητα τα δεδομένα χωρίς να διατρέξουμε όλο το ιστορικό των πράξεων του νοσοκομείου.

4. CREATE INDEX idx_praxis_aithousa ON ΙΑΤΡΙΚΕΣ_ΠΡΑΞΕΙΣ_ΑΣΘΕΝΗ (ΑΙΘΟΥΣΑ, ΗΜΕΡΟΜΗΝΙΑ_ΩΡΑ); Εδώ έχουμε ένα σύνθετο ευρετήριο (composite index) το οποίο είναι εξαιρετικά κρίσιμο για τη διασφάλιση της ορθότητας των δεδομένων μέσω των Triggers. Κατά την εισαγωγή μιας νέας επέμβασης, ο trigger trg_no_concurrent_operation πρέπει να ελέγξει αν η συγκεκριμένη αίθουσα είναι κατειλημμένη τη συγκεκριμένη ώρα. Το ευρετήριο αυτό βοηθά να βρεθούν αμέσως οι εγγραφές που πληρούν τα κριτήρια του χώρου και του χρόνου, κάνοντας τον έλεγχο αλληλεπικάλυψης αστραπιαίο.

5. CREATE INDEX idx_allerg_ousia ON ΑΛΛΕΡΓΙΕΣ (ΔΡΑΣΤΙΚΗ_ΟΥΣΙΑ); Αυτό το index δημιουργήθηκε για να υποστηρίξει τον κρίσιμο κανόνα ασφαλείας κατά τη συνταγογράφηση (trg_no_allergy_prescription). Κάθε φορά που ένας ιατρός γράφει ένα φάρμακο, το σύστημα ελέγχει αν ο ασθενής είναι αλλεργικός στη δραστική ουσία. Το ευρετήριο επιταχύνει τις συνενώσεις (JOIN) μεταξύ των πινάκων ΑΛΛΕΡΓΙΕΣ και ΦΑΡΜΑΚΑ_ΑΡΘΡΟ_57 βάσει της δραστικής ουσίας, εξασφαλίζοντας μηδενική καθυστέρηση στη λειτουργία του νοσοκομείου.

6. CREATE INDEX idx_diag_icd10 ON ΔΙΑΓΝΩΣΕΙΣ_ΝΟΣΗΛΕΙΑΣ (ΚΩΔΙΚΟΣ_ICD10); Σε ερωτήματα όπου αναζητούμε ασθενείς ή αναλύουμε νοσηλείες βάσει συγκεκριμένης πάθησης ή ομάδας παθήσεων (π.χ. ασθενείς με COVID-19 ή καρδιακά νοσήματα), αυτό το ευρετήριο βοηθά να βρεθεί αμέσως ο κωδικός ICD-10. Επιταχύνει τα WHERE ΚΩΔΙΚΟΣ_ICD10 = '...' και διευκολύνει την ομαδοποίηση (GROUP BY ΚΩΔΙΚΟΣ_ICD10) για την παραγωγή επιδημιολογικών στατιστικών.

7. CREATE INDEX idx_vardies_tmima_date ON ΒΑΡΔΙΕΣ (ΟΝΟΜΑ_ΤΜΗΜΑΤΟΣ, ΗΜΕΡΟΜΗΝΙΑ); Το συγκεκριμένο σύνθετο ευρετήριο είναι ιδανικό για ερωτήματα που σχετίζονται με τον προγραμματισμό και την προβολή του ωραρίου. Όταν θέλουμε να βρούμε τις βάρδιες ενός συγκεκριμένου τμήματος για μια συγκεκριμένη μέρα (π.χ. WHERE ΟΝΟΜΑ_ΤΜΗΜΑΤΟΣ = 'Παθολογικό' AND ΗΜΕΡΟΜΗΝΙΑ = '2024-05-20'), αυτό το index αποτρέπει τη σάρωση όλου του πίνακα των βαρδιών και επιστρέφει απευθείας το ημερήσιο πρόγραμμα.

8. CREATE INDEX idx_axiol_iatros ON ΑΞΙΟΛΟΓΗΣΗ_ΙΑΤΡΟΥ (ΑΜΚΑ_ΙΑΤΡΟΥ); Στα ερωτήματα όπου μας ζητείται να βρούμε τη μέση βαθμολογία αξιολόγησης ανά ιατρό ή να αναδείξουμε τους καλύτερους/χειρότερους ιατρούς βάσει της ποιότητας φροντίδας, αυτό το ευρετήριο μας είναι απαραίτητο. Επιταχύνει τις aggregate functions (AVG(ΠΟΙΟΤΗΤΑ_ΙΑΤΡΙΚΗΣ_ΦΡΟΝΤΙΔΑΣ)) και τη λειτουργία GROUP BY ΑΜΚΑ_ΙΑΤΡΟΥ, καθώς συγκεντρώνει ταχύτητα όλες τις κριτικές που αντιστοιχούν στον εκάστοτε ιατρό.

9. CREATE INDEX idx_nosilia_asthenis ON ΝΟΣΗΛΕΙΑ (ΑΜΚΑ_ΑΣΘΕΝΗ); Αυτό το ευρετήριο είναι θεμελιώδες για την αναζήτηση του ιατρικού ιστορικού ενός ασθενούς. Σε κάθε ερώτημα όπου μας ζητείται να βρούμε τις νοσηλείες, το κόστος ή τη συνολική διάρκεια παραμονής ενός συγκεκριμένου ατόμου (π.χ. WHERE ΑΜΚΑ_ΑΣΘΕΝΗ = '...'), το index επιταχύνει δραματικά την ανάκτηση των δεδομένων χωρίς να χρειαστεί να διατρέξουμε ολόκληρο τον πίνακα των νοσηλείων του νοσοκομείου.

10. CREATE INDEX idx_omada_amka ON ΟΜΑΔΑ_ΕΦΗΜΕΡΙΑΣ (ΑΜΚΑ_ΠΡΟΣΩΠΙΚΟΥ); Το ευρετήριο αυτό είναι καθοριστικής σημασίας για την εύρυθμη

και ταχύτατη εκτέλεση των Triggers που ελέγχουν τα ωράρια του προσωπικού (π.χ. trg_omada_monthly_limit και trg_omada_8hour_rest). Καθώς το σύστημα πρέπει να υπολογίσει άμεσα πόσες βάρδιες έχει κάνει ο υπάλληλος τον τρέχοντα μήνα ή πότε τελείωσε η προηγούμενη βάρδιά του, το ευρετήριο συγκεντρώνει αμέσως όλες τις συμμετοχές του συγκεκριμένου ΑΜΚΑ, εξασφαλίζοντας ότι η εισαγωγή νέας βάρδιας (INSERT) δεν θα καθυστερήσει.

11. CREATE INDEX idx_iatros_epoptis ON ΙΑΤΡΟΙ (ΑΜΚΑ_ΕΠΟΠΤΗ); Στο σύστημά μας έχουμε ορίσει περιορισμούς (όπως ο trigger trg_iatros_no_cycle_insert) οι οποίοι διατρέχουν αναδρομικά το ιεραρχικό δέντρο εποπτείας των ιατρών για να αποτρέψουν κυκλικές αναθέσεις (π.χ. ένας ιατρός να επιβλέπει έμμεσα τον εαυτό του). Το ευρετήριο στο ΑΜΚΑ_ΕΠΟΠΤΗ επιταχύνει τις αλληπάλληλες αναδρομικές αναζητήσεις (SELECT ... WHERE ΑΜΚΑ_ΙΑΤΡΟΥ = v_current), μειώνοντας κατακόρυφα τον υπολογιστικό φόρτο.

12. CREATE INDEX idx_nosilia_eisagogi ON ΝΟΣΗΛΕΙΑ (ΗΜΕΡΟΜΗΝΙΑ_ΕΙΣΑΓΩΓΗΣ); Σε πολλά ερωτήματα διοικητικής ή οικονομικής φύσεως (reporting), μας ζητείται να αναζητήσουμε, να ομαδοποιήσουμε ή να καταμετρήσουμε νοσηλείες βάσει της χρονικής περιόδου διεξαγωγής τους. Το ευρετήριο στην ημερομηνία εισαγωγής κάνει πολύ πιο αποδοτικά τα ερωτήματα που αναζητούν περιστατικά μέσα σε ένα συγκεκριμένο εύρος ημερομηνιών (π.χ. WHERE ΗΜΕΡΟΜΗΝΙΑ_ΕΙΣΑΓΩΓΗΣ BETWEEN ... AND ...).

13. CREATE INDEX idx_prosop_typos ON ΠΡΟΣΩΠΙΚΟ (ΤΥΠΟΣ_ΠΡΟΣΩΠΙΚΟΥ); Αυτό το ευρετήριο είναι χρήσιμο καθώς σε ερωτήματα (αλλά και σε Triggers, όπως ο έλεγχος βαρδιών) χρειαζόμαστε άμεση πρόσβαση στην κατηγορία του προσωπικού. Επιταχύνει το φίλτρο WHERE ΤΥΠΟΣ_ΠΡΟΣΩΠΙΚΟΥ = 'Νοσηλεύτης', εντοπίζοντας γρήγορα μόνο τα άτομα της συγκεκριμένης κατηγορίας, χωρίς να σαρώνεται ολόκληρο το μητρώο του νοσοκομείου.

14. CREATE INDEX idx_fa57_ousia ON ΦΑΡΜΑΚΑ_ΑΡΘΡΟ_57 (ΔΡΑΣΤΙΚΗ_ΟΥΣΙΑ); Το ευρετήριο αυτό λειτουργεί συνδυαστικά με το idx_allerg_ousia για την αποτροπή ιατρικών λαθών κατά τη συνταγογράφηση. Επειδή ο πίνακας αυτός συνδέει κάθε φάρμακο με τις δραστικές του ουσίες, το index κάνει πολύ πιο αποδοτικό το JOIN μεταξύ της αλλεργίας του ασθενή και της χημικής σύνθεσης του φαρμάκου.

15. CREATE INDEX idx_syntago_nosilia ON ΣΥΝΤΑΓΟΓΡΑΦΗΣΗ (ΚΩΔΙΚΟΣ_ΝΟΣΗΛΕΙΑΣ); Αυτό το index είναι κατάλληλο για την αναζήτηση της φαρμακευτικής αγωγής που χορηγήθηκε αποκλειστικά κατά τη διάρκεια μιας συγκεκριμένης νοσηλείας. Επιπρόσθετα, υποστηρίζει άμεσα τον Trigger trg_eval_doctor_prescribed, ο οποίος πρέπει να ελέγξει ταχύτατα αν ο ιατρός που πάει να αξιολογήσει ο ασθενής, του είχε όντως συνταγογραφήσει φάρμακα κατά τη συγκεκριμένη παραμονή του στο νοσοκομείο.

16. CREATE INDEX idx_iatros_vathmida ON ΙΑΤΡΟΙ (ΒΑΘΜΙΔΑ); Στα ερωτήματα όπου πρέπει να αναζητήσουμε, να ταξινομήσουμε ή να μετρήσουμε ιατρούς βάσει της ιεραρχίας τους (π.χ. αναζήτηση μόνο των "Ειδικευόμενων"), αυτό το ευρετήριο κάνει την προσπέλαση άμεση. Επιπλέον, βοηθά τους εσωτερικούς ελέγχους της βάσης (Triggers), όπως όταν το σύστημα ελέγχει αν σε μια εφημερία υπάρχει παρών τουλάχιστον ένας ανώτερος ιατρός (WHERE ΒΑΘΜΙΔΑ IN ('Επιμελητής Α', 'Διευθυντής')).

17. CREATE INDEX idx_nosilia_tmima ON ΝΟΣΗΛΕΙΑ (ΟΝΟΜΑ_ΤΜΗΜΑΤΟΣ); Το ευρετήριο αυτό είναι απαραίτητο για την εξαγωγή στατιστικών δεδομένων που αφορούν τη λειτουργία κάθε κλινικής ξεχωριστά. Σε ερωτήματα που απαιτούν τον υπολογισμό της τρέχουσας πληρότητας ενός τμήματος, των συνολικών εσόδων του ή του μέσου όρου ημερών νοσηλείας ανά κλινική (WHERE ΟΝΟΜΑ_ΤΜΗΜΑΤΟΣ = '...'), αυτό το index αποτρέπει το full table scan του τεράστιου πίνακα των νοσηλείων.

18. CREATE INDEX idx_nosilia_exodos ON ΝΟΣΗΛΕΙΑ (ΗΜΕΡΟΜΗΝΙΑ_ΕΞΟΔΟΥ); Αντίστοιχα με την ημερομηνία εισαγωγής, αυτό το ευρετήριο βελτιστοποιεί τα ερωτήματα που αναζητούν αποκλειστικά ολοκληρωμένες νοσηλείες (π.χ. ασθενείς που πήραν εξιτήριο μέσα σε έναν συγκεκριμένο μήνα). Επιπλέον, υποστηρίζει την άμεση εκτέλεση του trigger trg_eval_admission_complete, ο οποίος ελέγχει ταχύτητα αν υπάρχει καταχωρημένη ημερομηνία εξόδου προτού επιτρέψει την αξιολόγηση μιας νοσηλείας.

19. CREATE INDEX idx_syntago_iatros ON ΣΥΝΤΑΓΟΓΡΑΦΗΣΗ (ΑΜΚΑ_ΙΑΤΡΟΥ); Σε ερωτήματα ελέγχου και διοικητικής αναφοράς, συχνά ζητείται να ανακτηθεί το ιστορικό συνταγογραφήσεων ενός συγκεκριμένου ιατρού. Το συγκεκριμένο ευρετήριο συγκεντρώνει γρήγορα όλες τις συνταγές που έχει εκδώσει ο εκάστοτε ιατρός, επιταχύνοντας τις αναζητήσεις και τα GROUP BY ΑΜΚΑ_ΙΑΤΡΟΥ.

20. CREATE INDEX idx_syntago_farmako ON ΣΥΝΤΑΓΟΓΡΑΦΗΣΗ (ΦΑΡΜΑΚΟ_ID); Το ευρετήριο αυτό είναι κρίσιμο για τη φαρμακοεπαγρύπνηση και την ανάλυση της κατανάλωσης φαρμάκων. Όταν το νοσοκομείο θέλει να εξάγει αναφορές για το πόσο συχνά συνταγογραφήθηκε ένα συγκεκριμένο σκεύασμα μέσα σε ένα έτος (WHERE ΦΑΡΜΑΚΟ_ID = ...), το index επιστρέφει ταχύτητα τις αντίστοιχες εγγραφές.

21. CREATE INDEX idx_exam_asthenis ON ΕΞΕΤΑΣΕΙΣ_ΑΠΟΤΕΛΕΣΜΑΤΑ (ΑΜΚΑ_ΑΣΘΕΝΗ); Αποτελεί ένα από τα πιο νευραλγικά ευρετήρια για τον ηλεκτρονικό φάκελο του ασθενούς. Επιτρέπει στον ιατρό να ανακτήσει αστραπιαία ολόκληρο το ιστορικό των διαγνωστικών και κλινικών εξετάσεων ενός συγκεκριμένου ασθενούς, συνδέοντας άμεσα το ΑΜΚΑ με τα αποτελέσματα, χωρίς να σαρώνεται η παγκόσμια βάση εξετάσεων του νοσοκομείου.

22. CREATE INDEX idx_exam_iatros ON ΕΞΕΤΑΣΕΙΣ_ΑΠΟΤΕΛΕΣΜΑΤΑ (ΑΜΚΑ_ΙΑΤΡΟΥ); Χρησιμοποιείται για να εντοπίζονται γρήγορα όλες οι εξετάσεις που έχει διατάξει ή επιβλέψει ένας συγκεκριμένος ιατρός. Αυτό είναι ιδιαίτερα χρήσιμο για την αξιολόγηση του φόρτου εργασίας των ιατρών, καθώς και για ιατρονομικούς ελέγχους των παραπτεμπτικών.

23. CREATE INDEX idx_exam_nosilia ON ΕΞΕΤΑΣΕΙΣ_ΑΠΟΤΕΛΕΣΜΑΤΑ (ΚΩΔΙΚΟΣ_ΝΟΣΗΛΕΙΑΣ); Το ευρετήριο αυτό ομαδοποιεί τις εξετάσεις με βάση τη νοσηλεία. Στα ερωτήματα όπου αναλύεται το κόστος μιας συγκεκριμένης εισαγωγής ή ζητείται το πλήρες ιατρικό ανακοινωθέν κατά τη διάρκεια της παραμονής του ασθενούς στο νοσοκομείο, η αναζήτηση μέσω του κωδικού νοσηλείας γίνεται άμεσα αποδοτική.

24. CREATE INDEX idx_praxis_iatros ON ΙΑΤΡΙΚΕΣ_ΠΡΑΞΕΙΣ_ΑΣΘΕΝΗ (ΑΜΚΑ_ΙΑΤΡΟΥ); Όπως ακριβώς χρειαζόμαστε το ιστορικό του ασθενούς, έτσι απαιτείται και η παρακολούθηση του κλινικού έργου του ιατρού. Το ευρετήριο αυτό βελτιστοποιεί τα ερωτήματα που αναζητούν σε πόσες και ποιες ιατρικές/χειρουργικές πράξεις συμμετείχε

ένας συγκεκριμένος ιατρός, επιταχύνοντας τον έλεγχο απόδοσης (WHERE AMKA_IATPOY = '...').

25. CREATE INDEX idx_praxis_nosilia ON ΙΑΤΡΙΚΕΣ_ΠΡΑΞΕΙΣ_ΑΣΘΕΝΗ

(ΚΩΔΙΚΟΣ_ΝΟΣΗΛΕΙΑΣ); Αυτό το index βοηθά στην άμεση σύνδεση των ιατρικών πράξεων (π.χ. χειρουργεία, βιοψίες) με τη συγκεκριμένη νοσηλεία κατά την οποία πραγματοποιήθηκαν. Είναι απαραίτητο για τη σωστή και γρήγορη εξαγωγή της αναλυτικής οικονομικής χρέωσης (billing) του ασθενούς κατά το εξιτήριο.

26. CREATE INDEX idx_tep_asthenis ON ΤΜΗΜΑ_ΕΠΕΙΓΟΝΤΩΝ_ΠΕΡΙΣΤΑΤΙΚΩΝ

(ΑΜΚΑ_ΑΣΘΕΝΗ); Όταν ένας ασθενής προσέρχεται στα Επείγοντα, το ιατρικό προσωπικό πρέπει να γνωρίζει άμεσα αν έχει νοσηλευτεί ξανά στο παρελθόν στην ίδια μονάδα (frequent flyers). Το ευρετήριο αυτό επιταχύνει την ανάκτηση του ιστορικού επισκέψεων ενός ατόμου στο ΤΕΠ, διευκολύνοντας την ταχύτερη λήψη ιατρικών αποφάσεων.

27. CREATE INDEX idx_asfal_asthenis ON ΑΣΦΑΛΙΣΤΙΚΟΣ_ΦΟΡΕΑΣ

(ΑΜΚΑ_ΑΣΘΕΝΗ); Στα ερωτήματα τιμολόγησης και διαχείρισης λογιστηρίου, είναι αναγκαία η άμεση εύρεση του ασφαλιστικού φορέα (π.χ. ΕΟΠΥΥ ή Ιδιωτική Ασφάλεια) που καλύπτει τον ασθενή. Το ευρετήριο αυτό καθιστά ταχύτατα τα JOIN ανάμεσα στα στοιχεία του ασθενούς, τις νοσηλείες του και το ασφαλιστικό του συμβόλαιο.

Fake Data

Τα fake data δημιουργήθηκαν μέσω του script “databases.py”, το οποίο είναι υλοποιημένο σε γλώσσα Python. Στο συγκεκριμένο script, χρησιμοποιήσαμε τις βιβλιοθήκες pandas, faker και random για την παραγωγή τυχαίων δεδομένων για κάθε πίνακα του σχεσιακού μοντέλου (Relational Model). Για τα δεδομένα, τα οποία δεν μπορούσαμε να φτιάξουμε με την faker (π.χ ονόματα τμημάτων), δημιουργήσαμε custom data pools με αληθοφανή δεδομένα, τα οποία αξιοποιήσαμε με κατάλληλες συναρτήσεις (π.χ random.choice()). Τα δεδομένα που δημιουργήσαμε αποθηκεύονται σε ξεχωριστά αρχεία μορφής CSV, με ένα διαφορετικό αρχείο να αντιστοιχεί σε κάθε πίνακα.

Για την ομαλή και ταχύτατη εισαγωγή των δεδομένων από τα αρχεία CSV στο Σύστημα Διαχείρισης Βάσεων Δεδομένων, αναπτύξαμε ένα εξειδικευμένο SQL script. Στο script αυτό επιλέχθηκε η χρήση της εντολής LOAD DATA LOCAL INFILE, η οποία αποτελεί τη βέλτιστη πρακτική για την εισαγωγή μεγάλου όγκου εγγραφών, καθώς παρακάμπτει το σημαντικό υπολογιστικό κόστος που θα προκαλούσε η εκτέλεση χιλιάδων μεμονωμένων εντολών INSERT.

Κατά τη συγγραφή του κώδικα φόρτωσης, εφαρμόσαμε συγκεκριμένες τεχνικές για τη διασφάλιση της ορθότητας των δεδομένων:

- Προσωρινή Απενεργοποίηση Ξένων Κλειδιών (SET FOREIGN_KEY_CHECKS = 0): Πριν την έναρξη της φόρτωσης, οι έλεγχοι για τα Foreign keys απενεργοποιήθηκαν προσωρινά. Αυτή η στρατηγική επιλογή εξαλείφει τα προβλήματα αλληλεξάρτησης μεταξύ των πινάκων κατά τη φάση της εισαγωγής και επιταχύνει δραματικά τη διαδικασία. Μετά το πέρας της φόρτωσης, οι έλεγχοι ενεργοποιούνται ξανά (SET FOREIGN_KEY_CHECKS = 1), προστατεύοντας την ακεραιότητα της βάσης.

- Χειρισμός Κενών Τιμών (NULL Handling): Καθώς τα αρχεία CSV συχνά αναπαριστούν τα κενά πεδία ως άδειες συμβολοσειρές, χρησιμοποιήθηκαν τοπικές μεταβλητές (π.χ. @εποπτης, @τιμη) σε συνδυασμό με τη συνάρτηση NULLIF(). Αυτό εξασφαλίζει ότι τα άδεια κελιά του CSV μεταφράζονται ορθά στην ειδική τιμή NULL της SQL (π.χ. SET AMKA_ΕΠΟΠΤΗ = NULLIF(@εποπτης, "")).
- Προσωρινή Απενεργοποίηση Triggers (SET @DISABLE_TRIGGERS = 1): Κατά τη μαζική φόρτωση πινάκων με αυστηρούς κανόνες ελέγχου (π.χ. ο πίνακας ΟΜΑΔΑ_ΕΦΗΜΕΡΙΑΣ), χρησιμοποιήθηκε μια global μεταβλητή για την παράκαμψη των Triggers. Αυτό κρίθηκε αναγκαίο για την αποφυγή καθυστερήσεων από διαρκείς ελέγχους ωραρίων και βαρδιών κατά την αρχική τροφοδότηση του συστήματος, εφόσον τα παραγόμενα CSV δεδομένα ήταν ήδη προ-ελεγμένα ως προς την εγκυρότητά τους.
- Ορισμός Διαχωριστών (Delimiters): Για κάθε αρχείο ορίστηκαν ρητά οι χαρακτήρες διαχωρισμού πεδίων (',' ή ';') και οι χαρακτήρες περικλεισμού (""), διασφαλίζοντας τη σωστή ανάγνωση ακόμα και αν τα ίδια τα δεδομένα περιείχαν κόμματα (π.χ. στις διευθύνσεις ή τα συμπτώματα των ασθενών).

Στη συνέχεια, θα αναλύσουμε τα δεδομένα που δημιουργήσαμε για τη βάση του νοσοκομείου για τη διετία 2024-2026.

Αρχικά, εισάγουμε 15 διαφορετικά ιατρικά τμήματα (όπως Καρδιολογία, Χειρουργική, ΜΕΘ κ.λπ.), το καθένα με τη δική του ιατρική περιγραφή, και αντιστοιχίζουμε σε αυτά συνολικά 300 κλίνες διαφόρων τύπων (20 ανά τμήμα). Έπειτα, προχωράμε στην εισαγωγή του προσωπικού, προσθέτοντας 650 υπαλλήλους: 300 ιατρούς (με κατανομή σε 4 ιεραρχικές βαθμίδες και 15 ειδικότητες), 250 νοσηλευτές και 100 διοικητικούς υπαλλήλους. Για κάθε μέλος του προσωπικού αναπτύξαμε εξειδικευμένες συναρτήσεις που παράγουν ρεαλιστικά, μοναδικά AMKA (με βάση την ημερομηνία γέννησης) και μοναδικά εταιρικά emails. Παράλληλα, εξασφαλίσαμε την ιεραρχική δομή του νοσοκομείου αναθέτοντας υποχρεωτικά επόπτες (Επιμελητές ή Διευθυντές) στους Ειδικευόμενους ιατρούς.

Στη συνέχεια, εισάγουμε 20.000 ασθενείς, μαζί με τα στοιχεία των οικείων τους και τα ασφαλιστικά τους ταμεία. Για την ιατρική τους φροντίδα, δημιουργήσαμε 5.000 επείγοντα περιστατικά (ΤΕΠ) και 30.000 νοσηλείες. Οι νοσηλείες συνδέονται με πραγματικούς κωδικούς ΚΕΝ, οι οποίοι αντιστοιχίζονται δυναμικά στο κατάλληλο τμήμα βάσει του Τομέα Κατηγορίας Ασθένειας (ΤΚΑ), εξασφαλίζοντας τον ρεαλιστικό υπολογισμό της διάρκειας και του κόστους κάθε νοσηλείας. Επιπρόσθετα, καταγράφουμε 8.000 ιατρικές πράξεις, εφαρμόζοντας αλγόριθμους αποτροπής χρονικής αλληλεπικάλυψης, ώστε να διασφαλίζεται ότι ο ίδιος ιατρός ή η ίδια αίθουσα δεν απασχολούνται ταυτόχρονα σε δύο διαφορετικές επεμβάσεις.

Το ιατρικό ιστορικό των ασθενών πλαισιώνεται περαιτέρω με 15.000 κλινικές εξετάσεις, διαγνώσεις νοσηλειών βάσει πραγματικών κωδικών ICD-10, 3.000 συνταγογραφήσεις, καθώς και τυχαία κατανομή αλλεργιών σε συγκεκριμένες δραστικές ουσίες, ελέγχοντας πάντα τους αντίστοιχους ιατρικούς περιορισμούς. Επίσης, προσομοιώσαμε ένα σύστημα αξιολόγησης, όπου περίπου το 40-50% των νοσηλειών και ιατρών λαμβάνουν βαθμολογίες για την ποιότητα των υπηρεσιών.

Τέλος, παράγουμε το πρόγραμμα εφημεριών του προσωπικού για 730 συνεχόμενες ημέρες (3 βάρδιες ημερησίως ανά τμήμα). Για τη δημιουργία των βαρδιών, ο αλγόριθμος ελέγχει και τηρεί αυστηρούς επιχειρησιακούς κανόνες, όπως το μηνιαίο όριο βαρδιών ανά τύπο υπαλλήλου, το ελάχιστο όριο δωρης ανάπαυσης, την αποτροπή 3 συνεχόμενων νυχτερινών

βαρδιών, αλλά και την υποχρεωτική παρουσία ανώτερου ιατρού στη βάρδια των Ειδικευόμενων.

Κοστολογηση και Φάρμακα

Η οντότητα ΚΟΣΤΟΛΟΓΗΣΗ δομείται από τα πεδία: KEN_ID, ΠΕΡΙΓΡΑΦΗ, ΚΟΣΤΟΣ_ΑΝΑ_ΚΕΝ, ΚΩΔΙΚΟΣ_ΚΕΝ και ΜΔΝ (Μέση Διάρκεια Νοσηλείας). Στο αρχείο με τους κωδικούς ΚΕΝ πολλές παθήσεις αντιστοιχούν στο ίδιο ΚΕΝ και συνεπώς ο κωδικός αυτός δεν μπορεί να χρησιμοποιηθεί ως πρωτεύον κλειδί. Ως βέλτιστη σχεδιαστική λύση, προστέθηκε το πεδίο KEN_ID ως ένα μοναδικό, auto-increment, το οποίο αναλαμβάνει αποκλειστικά τον ρόλο του Primary Key για τον συγκεκριμένο πίνακα.

Στον πίνακα ΦΑΡΜΑΚΑ, προέκυψε η σχεδιαστική ανάγκη να αποτυπωθεί το γεγονός ότι ένα συγκεκριμένο φάρμακο μπορεί να παρασκευάζεται σε διάφορες εκδοχές (με διαφορετικούς συνδυασμούς δραστικών ουσιών). Για αυτό ορίσαμε ως Πρωτεύον Κλειδί το πεδίο ΦΑΡΜΑΚΟ_ID με auto-increment. Επιπλέον, εισήχθη το χαρακτηριστικό ΣΥΝΤΑΓΗ, το οποίο μέσω ακεραίων τιμών (1, 2, ...) διαφοροποιεί και υποδηλώνει τη συγκεκριμένη φαρμακευτική εκδοχή του σκευάσματος. Με αυτόν τον τρόπο διασφαλίζεται η μέγιστη ακρίβεια στους ελέγχους για πιθανές αλλεργίες, καθώς το σύστημα αναγνωρίζει μέσω του ΦΑΡΜΑΚΟ_ID ότι μια αλλεργία μπορεί να σχετίζεται αποκλειστικά με μια συγκεκριμένη φόρμουλα του φαρμάκου, και όχι απαραίτητα με όλες τις εμπορικές του παραλλαγές

Ανεξαρτησία του πίνακα ΦΩΤΟΓΡΑΦΙΕΣ

Στο σχεσιακό διάγραμμα, ο πίνακας ΦΩΤΟΓΡΑΦΙΕΣ φαίνεται να μην συνδέεται με κανέναν άλλο πίνακα μέσω Foreign Keys. Επειδή στη βάση μας θέλουμε να αποθηκεύουμε φωτογραφίες για πολλές διαφορετικές οντότητες (όπως Ιατρούς, Τμήματα ή Εξοπλισμό) θα έπρεπε είτε να φτιάξουμε πολλούς διαφορετικούς πίνακες (π.χ. ΦΩΤΟΓΡΑΦΙΕΣ_ΙΑΤΡΩΝ, ΦΩΤΟΓΡΑΦΙΕΣ_ΤΜΗΜΑΤΩΝ), είτε έναν πίνακα με πολλά άδεια πεδία (NULL), σπαταλώντας χώρο.

Αντί για αυτό, επιλέξαμε να χρησιμοποιώντας δύο δυναμικά πεδία:

- ΤΥΠΟΣ_ΟΝΤΟΤΗΤΑΣ: Δείχνει σε ποια κατηγορία ανήκει η φωτογραφία (π.χ. "Ιατρός" ή "Τμήμα").
- ID_ΟΝΤΟΤΗΤΑΣ: Αποθηκεύει το αντίστοιχο Πρωτεύον Κλειδί (π.χ. το ΑΜΚΑ του ιατρού ή το όνομα του τμήματος). Επειδή πρέπει να δέχεται και αριθμούς και γράμματα, τον ορίσαμε ως απλό κείμενο (VARCHAR).

Με αυτόν τον τρόπο, ο πίνακας παραμένει "ανεξάρτητος" στο διάγραμμα. Η βάση μας κερδίζει τεράστια επεκτασιμότητα, καθώς στο μέλλον μπορούμε να ανεβάσουμε φωτογραφίες για οποιαδήποτε άλλη οντότητα (π.χ. Ασθενείς), χωρίς να χρειαστεί να αλλάξουμε καθόλου τη δομή της βάσης δεδομένων.

Q04 — Ανάλυση EXPLAIN και FORCE INDEX

(α) Αποτέλεσμα EXPLAIN

Έκδοση χωρίς FORCE INDEX:

```
MariaDB [hospital]> source D:\DBLAB-2026\queries\Q04.sql
```

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	I	const	PRIMARY	PRIMARY	46	const	1	Using index
1	SIMPLE	Π	const	PRIMARY	PRIMARY	46	const	1	
1	SIMPLE	AI	ref	idx_axiol_iatros	idx_axiol_iatros	46	const	18	Using where
1	SIMPLE	N	eq_ref	PRIMARY	PRIMARY	4	hospital.AI.ΚΩΔΙΚΟΣ_ΝΟΣΗΛΕΙΑΣ	1	Using where; Using index
1	SIMPLE	AN	eq_ref	PRIMARY,ΚΩΔΙΚΟΣ_ΝΟΣΗΛΕΙΑΣ	PRIMARY	50	hospital.AI.AMKA_ΑΣΘΕΝΗ,hospital.N.ΚΩΔΙΚΟΣ_ΝΟΣΗΛΕΙΑΣ	1	Using where

5 rows in set (0.054 sec)

Έκδοση με FORCE INDEX (idx_axiol_iatros):

Το query plan είναι πανομοιότυπο αφού ο optimizer επιλέγει τα ίδια indexes και στις δύο περιπτώσεις.

ΟΝΟΜΑ	ΕΠΩΝΥΜΟ	ΑΜΚΑ_ΙΑΤΡΟΥ	ΜΕΣΟΣ_ΟΡΟΣ_ΑΞΙΟΛΟΓΗΣΗΣ_ΙΑΤΡΟΥ	ΜΕΣΟΣ_ΟΡΟΣ_ΣΥΝΟΛΙΚΗΣ_ΕΜΠΕΙΡΙΑΣ
Ζαφείριος	Λαμπρινός	16059786000	2.61	3.50

1 row in set (0.007 sec)

ΟΝΟΜΑ	ΕΠΩΝΥΜΟ	ΑΜΚΑ_ΙΑΤΡΟΥ	ΜΕΣΟΣ_ΟΡΟΣ_ΑΞΙΟΛΟΓΗΣΗΣ_ΙΑΤΡΟΥ	ΜΕΣΟΣ_ΟΡΟΣ_ΣΥΝΟΛΙΚΗΣ_ΕΜΠΕΙΡΙΑΣ
Ζαφείριος	Λαμπρινός	16059786000	2.61	3.50

1 row in set (0.001 sec)

(β) Εναλλακτική έκδοση με FORCE INDEX

LEFT JOIN ΑΞΙΟΛΟΓΗΣΗ_ΙΑΤΡΟΥ AI FORCE INDEX (idx_axiol_iatros)
ON I.AMKA_ΙΑΤΡΟΥ = AI.AMKA_ΙΑΤΡΟΥ

Αποτέλεσμα και των δύο εκδόσεων:

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	I	const	PRIMARY	PRIMARY	46	const	1	Using index
1	SIMPLE	Π	const	PRIMARY	PRIMARY	46	const	1	
1	SIMPLE	AI	ref	idx_axiol_iatros	idx_axiol_iatros	46	const	18	Using where
1	SIMPLE	N	eq_ref	PRIMARY	PRIMARY	4	hospital.AI.ΚΩΔΙΚΟΣ_ΝΟΣΗΛΕΙΑΣ	1	Using where; Using index
1	SIMPLE	AN	eq_ref	PRIMARY,ΚΩΔΙΚΟΣ_ΝΟΣΗΛΕΙΑΣ	PRIMARY	50	hospital.AI.AMKA_ΑΣΘΕΝΗ,hospital.N.ΚΩΔΙΚΟΣ_ΝΟΣΗΛΕΙΑΣ	1	Using where

5 rows in set (0.001 sec)

Και οι δύο εκδόσεις επιστρέφουν τα ίδια αποτελέσματα, επιβεβαιώνοντας ότι το FORCE INDEX δεν αλλάζει τη σημασία του query.

(γ) Σύγκριση εκτιμώμενου κόστους και πραγματικού χρόνου εκτέλεσης

Εκτιμώμενο κόστος (EXPLAIN):

Και οι δύο εκδόσεις παράγουν ίδιο query plan. Ο optimizer επιλέγει αυτόματα το `idx_axiol_iatros` για τον πίνακα `ΑΞΙΟΛΟΓΗΣΗ_ΙΑΤΡΟΥ` με εκτιμώμενες 18 γραμμές (όπως φαίνεται στις παραπάνω εικόνες). Το `FORCE INDEX` δεν αλλάζει την επιλογή του.

Πραγματικός χρόνος εκτέλεσης (SHOW PROFILES):

Query_ID	Duration	Query
1	0.00757890	SELECT Π.ΟΝΟΜΑ, Π.ΕΠΩΝΥΜΟ, Ι.ΑΜΚΑ_ΙΑΤΡΟΥ, ROUND(AVG(ΑΙ.ΠΟΙΟΤΗΤΑ_ΙΑΤΡΙΚΗΣ_ΦΡΟΝΤΙΔΑΣ), 2) AS ΜΕΣΟΣ_ΟΡΟΣ_ΑΞΙΟΛΟΓΗΣΗΣ_ΙΑΤΡΟΥ, ROUND(AVG(ΑΝ.ΣΥΝΟΛΙΚΗ_ΕΜΠΕΙΡΙΑ), 2) AS ΜΕΣΟΣ_ΟΡΟΣ_ΣΥΝΟΛ
2	0.00198770	SELECT Π.ΟΝΟΜΑ, Π.ΕΠΩΝΥΜΟ, Ι.ΑΜΚΑ_ΙΑΤΡΟΥ, ROUND(AVG(ΑΙ.ΠΟΙΟΤΗΤΑ_ΙΑΤΡΙΚΗΣ_ΦΡΟΝΤΙΔΑΣ), 2) AS ΜΕΣΟΣ_ΟΡΟΣ_ΑΞΙΟΛΟΓΗΣΗΣ_ΙΑΤΡΟΥ, ROUND(AVG(ΑΝ.ΣΥΝΟΛΙΚΗ_ΕΜΠΕΙΡΙΑ), 2) AS ΜΕΣΟΣ_ΟΡΟΣ_ΣΥΝΟΛ

2 rows in set (0.001 sec)

Χωρίς `FORCE INDEX`: 0.00758 sec

Με `FORCE INDEX (idx_axiol_iatros)`: 0.00199 sec

Η έκδοση με `FORCE INDEX` είναι περίπου 4 φορές γρηγορότερη

(δ) Ανάλυση Συμπερασμάτων

Ο MariaDB optimizer επιλέγει αυτόματα το `idx_axiol_iatros` για την αναζήτηση στον πίνακα `ΑΞΙΟΛΟΓΗΣΗ_ΙΑΤΡΟΥ`, καθώς είναι ο πιο κατάλληλος για φιλτράρισμα με `ΑΜΚΑ_ΙΑΤΡΟΥ`. Το `EXPLAIN` plan είναι πανομοιότυπο και στις δύο εκδόσεις, επιβεβαιώνοντας ότι ο optimizer κάνει ήδη τη βέλτιστη επιλογή ως προς τα indexes. Η διαφορά στον πραγματικό χρόνο (0.00758 vs 0.00199 sec) οφείλεται στο query cache της MariaDB που αξιοποιήθηκε στη δεύτερη εκτέλεση. Συμπερασματικά, το `FORCE INDEX` δεν βελτιώνει το query plan αλλά η δεύτερη εκτέλεση επωφελείται από τον cache, γεγονός που υποδηλώνει ότι σε περιβάλλον παραγωγής με επαναλαμβανόμενα queries ο cache παίζει καθοριστικό ρόλο στην απόδοση.

Q06 — Ανάλυση EXPLAIN και FORCE INDEX

(α) Αποτέλεσμα EXPLAIN

Έκδοση χωρίς `FORCE INDEX`:

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	I	const	PRIMARY	PRIMARY	46	const	1	Using index
1	SIMPLE	Π	const	PRIMARY	PRIMARY	46	const	1	
1	SIMPLE	ΑΙ	ref	idx_axiol_iatros	idx_axiol_iatros	46	const	18	Using where
1	SIMPLE	Ν	eq_ref	PRIMARY	PRIMARY	4	hospital.ΑΙ.ΚΩΔΙΚΟΣ_ΝΟΣΗΛΕΙΑΣ	1	Using where; Using index
1	SIMPLE	ΑΝ	eq_ref	PRIMARY,ΚΩΔΙΚΟΣ_ΝΟΣΗΛΕΙΑΣ	PRIMARY	50	hospital.ΑΙ.ΑΜΚΑ_ΑΙΘΕΝΗ,hospital.Ν.ΚΩΔΙΚΟΣ_ΝΟΣΗΛΕΙΑΣ	1	Using where

5 rows in set (0.054 sec)

Έκδοση με FORCE INDEX (idx_nosilia_asthenis): Το query plan είναι πανομοιότυπο αφού ο optimizer επιλέγει τα ίδια indexes και στις δύο περιπτώσεις.

(β) Εναλλακτική έκδοση με FORCE INDEX

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	A	const	PRIMARY	PRIMARY	46	const	1	Using temporary; Using filesort
1	SIMPLE	N	ref	idx_nosilia_asthenis	idx_nosilia_asthenis	46	const	8	Using index condition; Using where; Using filesort
1	SIMPLE	A	ref	PRIMARY	PRIMARY	4	hospital.N.ΚΩΔΙΚΟΣ_ΝΟΣΗΛΕΙΑΣ	1	Using index
1	SIMPLE	K	eq_ref	PRIMARY	PRIMARY	42	hospital.A.ΚΩΔΙΚΟΣ_ICD10	1	Using where
1	SIMPLE	AN	eq_ref	PRIMARY,ΚΩΔΙΚΟΣ_ΝΟΣΗΛΕΙΑΣ	PRIMARY	50	const,hospital.N.ΚΩΔΙΚΟΣ_ΝΟΣΗΛΕΙΑΣ	1	

5 rows in set (0.004 sec)

JOIN ΝΟΣΗΛΕΙΑ Ν FORCE INDEX (idx_nosilia_asthenis)

ON A.AMKA_ΑΣΘΕΝΗ = N.AMKA_ΑΣΘΕΝΗ

Αποτέλεσμα και των δύο εκδόσεων:

AMKA_ΑΣΘΕΝΗ	ΟΝΟΜΑ	ΕΠΩΝΥΜΟ	ΚΩΔΙΚΟΣ_ΝΟΣΗΛΕΙΑΣ	ΗΜΕΡΟΜΗΝΙΑ_ΕΙΣΑΓΟΓΗΣ	ΗΜΕΡΟΜΗΝΙΑ_ΕΞΟΔΟΥ	ΚΩΣΤΟΣ_ΝΟΣΗΛΕΙΑΣ
22833194683	Μένανδρος	Ημερεδιδμος		12873 2026-08-15	2026-08-22	2541.00
22833194683	Μένανδρος	Ημερεδιδμος		3651 2026-06-30	2026-07-09	1827.00
22833194683	Μένανδρος	Ημερεδιδμος		13287 2026-06-15	2026-06-23	1494.40
22833194683	Μένανδρος	Ημερεδιδμος		3268 2026-05-23	2026-06-10	4636.80
22833194683	Μένανδρος	Ημερεδιδμος		26185 2025-07-09	2025-08-08	8302.80
22833194683	Μένανδρος	Ημερεδιδμος		21739 2025-03-15	2025-04-03	7728.71
22833194683	Μένανδρος	Ημερεδιδμος		6169 2025-01-12	2025-01-25	11487.67
22833194683	Μένανδρος	Ημερεδιδμος		14862 2024-07-02	2024-07-09	3591.00

8 rows in set (0.014 sec)

AMKA_ΑΣΘΕΝΗ	ΟΝΟΜΑ	ΕΠΩΝΥΜΟ	ΚΩΔΙΚΟΣ_ΝΟΣΗΛΕΙΑΣ	ΗΜΕΡΟΜΗΝΙΑ_ΕΙΣΑΓΟΓΗΣ	ΗΜΕΡΟΜΗΝΙΑ_ΕΞΟΔΟΥ	ΚΩΣΤΟΣ_ΝΟΣΗΛΕΙΑΣ
22833194683	Μένανδρος	Ημερεδιδμος		12873 2026-08-15	2026-08-22	2541.00
22833194683	Μένανδρος	Ημερεδιδμος		3651 2026-06-30	2026-07-09	1827.00
22833194683	Μένανδρος	Ημερεδιδμος		13287 2026-06-15	2026-06-23	1494.40
22833194683	Μένανδρος	Ημερεδιδμος		3268 2026-05-23	2026-06-10	4636.80
22833194683	Μένανδρος	Ημερεδιδμος		26185 2025-07-09	2025-08-08	8302.80
22833194683	Μένανδρος	Ημερεδιδμος		21739 2025-03-15	2025-04-03	7728.71
22833194683	Μένανδρος	Ημερεδιδμος		6169 2025-01-12	2025-01-25	11487.67
22833194683	Μένανδρος	Ημερεδιδμος		14862 2024-07-02	2024-07-09	3591.00

ΚΩΣΤΟΣ_ΝΟΣΗΛΕΙΑΣ	ΔΙΑΓΝΩΣΗ	ΚΩΣΤΟΣ_ΟΡΟΣ_ΑΞΙΟΛΟΓΗΣΗΣ
2541.00	Ενδοβλ: T22.2 - Κρυαγμα διάτρηση βελβή του καρπού και της άνωας χείρας	2.00
1827.00	Ενδοβλ: C24.0 - Χοληδόχου κύστος, μη καθιερωμένο Εξόδου: D27.7 - Άλλων οργάνων του πεπτικού συστήματος	2.00
1494.40	Ενδοβλ: G82.2 - Νοσητέλγεια του άνω άραου	2.00
4636.80	Ενδοβλ: G71 - Ελλειγματιμής ανακόλυας του άνω άραου Εξόδου: Q62.1 - Άρρητια και σπένωση του ουρητρου	NULL
8302.80	Ενδοβλ: S30* - Διατραμιας και μεταβολιας διατραμιας σε νοσάματα που ταξινομούνται ελλω Εξόδου: S13.5 - Διατραμιας στη θωρακιτική χείρα	1.00
7728.71	Ενδοβλ: L20.2 - Ερυθματιώθ διαματιμής ελ σπάρη που οφείλεται σε διαλύτας Εξόδου: Y40 - Αντεπαλματιμής και αντιπαρκατιμής φάρμακα	2.00
11487.67	Ενδοβλ: L50 - Άλλας διατραμιας του άραματος και του υποάραου αυτού που σχετίζονται με αατινοβόλια	NULL
3591.00	Ενδοβλ: Z58.0 - Εάωση σε άραμιο	NULL

ΚΩΣΤΟΣ_ΝΟΣΗΛΕΙΑΣ	ΔΙΑΓΝΩΣΗ	ΚΩΣΤΟΣ_ΟΡΟΣ_ΑΞΙΟΛΟΓΗΣΗΣ
2541.00	Ενδοβλ: T22.2 - Κρυαγμα διάτρηση βελβή του καρπού και της άνωας χείρας	2.00
1827.00	Ενδοβλ: C24.0 - Χοληδόχου κύστος, μη καθιερωμένο Εξόδου: D27.7 - Άλλων οργάνων του πεπτικού συστήματος	2.00
1494.40	Ενδοβλ: G82.2 - Νοσητέλγεια του άνω άραου	2.00
4636.80	Ενδοβλ: G71 - Ελλειγματιμής ανακόλυας του άνω άραου Εξόδου: Q62.1 - Άρρητια και σπένωση του ουρητρου	NULL
8302.80	Ενδοβλ: S30* - Διατραμιας και μεταβολιας διατραμιας σε νοσάματα που ταξινομούνται ελλω Εξόδου: S13.5 - Διατραμιας στη θωρακιτική χείρα	1.00
7728.71	Ενδοβλ: L20.2 - Ερυθματιώθ διαματιμής ελ σπάρη που οφείλεται σε διαλύτας Εξόδου: Y40 - Αντεπαλματιμής και αντιπαρκατιμής φάρμακα	2.00
11487.67	Ενδοβλ: L50 - Άλλας διατραμιας του άραματος και του υποάραου αυτού που σχετίζονται με αατινοβόλια	NULL
3591.00	Ενδοβλ: Z58.0 - Εάωση σε άραμιο	NULL

Και οι δύο εκδόσεις επιστρέφουν τα ίδια αποτελέσματα, επιβεβαιώνοντας ότι το FORCE INDEX δεν αλλάζει τη σημασία του query.

(γ) Σύγκριση εκτιμώμενου κόστους και πραγματικού χρόνου εκτέλεσης

Εκτιμώμενο κόστος (EXPLAIN):

Και οι δύο εκδόσεις παράγουν ίδιο query plan. Ο optimizer επιλέγει αυτόματα το `idx_nosilia_asthenis` για τον πίνακα `N` με εκτιμώμενες 8 γραμμές (όπως φαίνεται στη στήλη `rows` της εικόνας). Το `FORCE INDEX` δεν αλλάζει την επιλογή του.

Query_ID	Duration	Query
1	0.01399050	SELECT A.AMKA_AΣΘΕΝΗ, A.ΟΝΟΜΑ, A.ΕΠΩΝΥΜΟ, N.ΚΩΔΙΚΟΣ_ΝΟΣΗΛΕΙΑΣ, N.ΗΜΕΡΟΜΗΝΙΑ_ΕΙΣΑΓΩΓΗΣ, N.ΗΜΕΡΟΜΗΝΙΑ_ΕΞΟΔΟΥ, N.ΚΟΣΤΟΣ_ΝΟΣΗΛΕΙΑΣ, GROUP_CONCAT(DISTINCT CONCAT(Δ.ΤΥΠΟΣ, ': ', Κ.ΚΩΔ
2	0.00657480	SELECT A.AMKA_AΣΘΕΝΗ, A.ΟΝΟΜΑ, A.ΕΠΩΝΥΜΟ, N.ΚΩΔΙΚΟΣ_ΝΟΣΗΛΕΙΑΣ, N.ΗΜΕΡΟΜΗΝΙΑ_ΕΙΣΑΓΩΓΗΣ, N.ΗΜΕΡΟΜΗΝΙΑ_ΕΞΟΔΟΥ, N.ΚΟΣΤΟΣ_ΝΟΣΗΛΕΙΑΣ, GROUP_CONCAT(DISTINCT CONCAT(Δ.ΤΥΠΟΣ, ': ', Κ.ΚΩΔ

Πραγματικός χρόνος εκτέλεσης:

Χωρίς `FORCE INDEX`: 0.01399 sec

Με `FORCE INDEX (idx_nosilia_asthenis)`: 0.00657 sec

Η έκδοση με `FORCE INDEX` είναι περίπου 2 φορές γρηγορότερη.

(δ) Ανάλυση Συμπερασμάτων

Ο MariaDB optimizer επιλέγει αυτόματα το `idx_nosilia_asthenis` για την αναζήτηση στον πίνακα `ΝΟΣΗΛΕΙΑ`, καθώς είναι ο πιο κατάλληλος για φιλτράρισμα με `AMKA_AΣΘΕΝΗ`. Το `EXPLAIN` plan είναι πανομοιότυπο και στις δύο εκδόσεις, επιβεβαιώνοντας ότι ο optimizer ήδη κάνει τη βέλτιστη επιλογή ως προς την επιλογή index.

Ωστόσο, η έκδοση με `FORCE INDEX` εκτελέστηκε σε 0.00657 sec έναντι 0.01399 sec της αρχικής. Αυτή η διαφορά δεν οφείλεται στο βελτιωμένο πλάνο (αφού παρέμεινε ίδιο), αλλά στο query cache της MariaDB που αξιοποιήθηκε κατά τη δεύτερη συνεχόμενη εκτέλεση.

Επιπλέον, η εμφάνιση της ένδειξης `Using temporary; Using filesort` στον πίνακα `ΑΣΘΕΝΕΙΣ` στο `EXPLAIN` plan υποδηλώνει ότι η ταξινόμηση μέσω `ORDER BY` `ΗΜΕΡΟΜΗΝΙΑ_ΕΙΣΑΓΩΓΗΣ` δεν καλύπτεται από το υπάρχον index. Η δημιουργία ενός

σύνθετου index στη μορφή (ΑΜΚΑ_ΑΣΘΕΝΗ, ΗΜΕΡΟΜΗΝΙΑ_ΕΙΣΑΓΩΓΗΣ) θα μπορούσε να εξαλείψει την ανάγκη για filesort και να βελτιώσει περαιτέρω την απόδοση.

Διαχείριση Μεγάλου Όγκου Δεδομένων & Βελτιστοποίηση Εισαγωγής

Κατά το στήσιμο της βάσης, ένα από τα μεγαλύτερα πρακτικά προβλήματα που αντιμετωπίσαμε ήταν η μαζική εισαγωγή εκατοντάδων χιλιάδων εγγραφών. Ειδικά στον πίνακα ΟΜΑΔΑ_ΕΦΗΜΕΡΙΑΣ, η φόρτωση με τις αρχικές ρυθμίσεις ήταν πρακτικά αδύνατη: το σύστημα είτε εκτύπωνε σφάλματα χρονικού ορίου (timeouts), είτε χρειαζόταν πάνω από 20 λεπτά για να ολοκληρώσει τη διαδικασία.

Η τεράστια αυτή καθυστέρηση οφειλόταν κυρίως στο πολύ βαρύ υπολογιστικό κόστος των Triggers. Για κάθε μία από τις 200.000+ γραμμές, η βάση αναγκαζόταν να τρέχει βαριά ερωτήματα (όπως SELECT COUNT(*) και ελέγχους ημερομηνιών).

Για την λύση αυτού του προβλήματος, αλλάξαμε τα παρακάτω στο αρχείο "my.ini", το οποίο στο directory \xampp\mysql\bin\my.ini. :

max_allowed_packet = 512M (από 1M)

innodb_buffer_pool_size = 4G (από 16M)

innodb_log_file_size = 256M (από 5M)

innodb_log_buffer_size = 64M (από 8M)

innodb_flush_log_at_trx_commit = 0 (από 1)

Παράμετροι που προστέθηκαν (κάτω από το innodb_lock_wait_timeout=50):

bulk_insert_buffer_size = 1G

net_read_timeout = 7200

net_write_timeout = 7200

wait_timeout = 7200

interactive_timeout = 7200

innodb_doublewrite = 0

Σκοπός ήταν να παραχωρήσουμε στη βάση δεδομένων μας περισσότερη μνήμη RAM (ελαχιστοποιώντας έτσι τις χρονοβόρες εγγραφές/αναγνώσεις στον σκληρό δίσκο) και να αποτρέψουμε τις πρόωρες αποσυνδέσεις (timeouts) κατά τη φόρτωση μεγάλων αρχείων.

Επιπλέον, τα συνθετικά δεδομένα (Mock Data) που παραδίδονται στον φάκελο της εργασίας έχουν ήδη ελεγχθεί ως προς την εγκυρότητά τους με χρήση των Triggers. Για να αποφευχθεί

η υπερβολική καθυστέρηση (άνω των 10-15 λεπτών) κατά την μαζική εισαγωγή (Bulk Loading) των 250.000+ εγγραφών κατά την αξιολόγηση, έχουμε προσθέσει προσωρινή απενεργοποίηση των triggers στο αρχείο load.sql (μέσω της μεταβλητής SET @DISABLE_TRIGGERS = 1;)

Σε περίπτωση που επιθυμείτε να παραγάγετε ένα νέο, δικό σας σκετ δεδομένων εκτελώντας το script python databases.py και θέλετε να ελέγξετε την ακεραιότητά τους κατά τη φόρτωση (ώστε οι Triggers να απορρίψουν τυχόν ασυνέπειες του τυχαίου αλγορίθμου, όπως παραβιάσεις 8ώρου ή μηνιαίου ορίου βαρδιών), θα πρέπει να κάνετε την εξής αλλαγή στο αρχείο load.sql:

Ενεργοποίηση των Triggers:

Εντοπίστε και διαγράψτε (ή σχολιάστε με --) τις εξής δύο γραμμές που βρίσκονται πριν και μετά την εντολή εισαγωγής του πίνακα ΟΜΑΔΑ_ΕΦΗΜΕΡΙΑΣ:

```
SET @DISABLE_TRIGGERS = 1;
```

```
SET @DISABLE_TRIGGERS = 0;
```

Με την αφαίρεση αυτών των εντολών, ο μηχανισμός παράκαμψης απενεργοποιείται και οι Triggers θα εκτελούνται πλέον για κάθε εγγραφή ξεχωριστά, επιβάλλοντας αυστηρά όλους τους επιχειρησιακούς κανόνες (π.χ. έλεγχος 8ώρου, όρια βαρδιών).

Σημείωση για τη Φόρτωση Δεδομένων (Data Loading & Validation):

Η μαζική εισαγωγή των δεδομένων (load.sql) έχει ρυθμιστεί με χρήση IGNORE INTO TABLE. Αυτό διασφαλίζει ότι η διαδικασία φόρτωσης θα ολοκληρωθεί ακόμα και αν υπάρχουν σπάνιες περιπτώσεις δεδομένων που παραβιάζουν τους κανόνες ακεραιότητας των Triggers, καθώς το σύστημα θα παρακάμπτει αυτόματα τις προβληματικές γραμμές, καταγράφοντάς τις ως Warnings, χωρίς να διακόπτεται η εκτέλεση του script.

Εάν δημιουργήσετε δικό σας σκετ δεδομένων μέσω του databases.py και επιθυμείτε η φόρτωση να λειτουργεί σε Strict Mode, δηλαδή να διακόπτεται η διαδικασία αμέσως μόλις εντοπιστεί οποιαδήποτε παραβίαση κανόνα από τους Triggers, θα πρέπει να αφαιρέσετε τη λέξη IGNORE μπροστά από το INTO TABLE, από τις αντίστοιχες εντολές LOAD DATA LOCAL INFILE μέσα στο αρχείο load.sql. Με την αφαίρεση του IGNORE, το σύστημα θα απορρίπτει ολόκληρη τη φόρτωση του πίνακα μόλις εντοπιστεί το πρώτο σφάλμα ακεραιότητας, επιτρέποντάς σας να εντοπίσετε και να διορθώσετε άμεσα τα προβληματικά δεδομένα.

AI/LLMS

Για την εργασία μας, χρησιμοποιήσαμε ως βοηθητικό εργαλείο το LLM Gemini. Η χρήση του αφορούσε αποκλειστικά την κατανόηση των βιβλιοθηκών Faker και Random της Python και

τη συγγραφή ορισμένων τμημάτων του κώδικα για την παραγωγή των συνθετικών δεδομένων.